MBCODE

Guide d'installation

Application de gestion MBCODE

Location d'espaces · Camps de formation · Formations IA · Facturation · Trésorerie

MBCODE — Quartier Makepe, Douala, Cameroun
infos@mbcode.cm · +237 6 59 17 27 62

Sommaire

1. Introduction
2. Prérequis
3. Installation avec Docker (recommandée)
4. Installation manuelle (sans Docker)
5. Configuration des variables d'environnement
6. Première connexion et création des comptes
7. Jeu de données de démonstration
8. Tâche planifiée : génération des charges fixes
9. Sauvegarde et mise à jour
10. Dépannage
11. Support

1. Introduction

Ce document décrit les étapes nécessaires à l'installation et à la mise en service de l'application de gestion MBCODE, aussi bien via Docker (méthode recommandée) que par une installation manuelle sur un serveur disposant de Python et PostgreSQL. Il s'adresse à toute personne chargée du déploiement ou de la maintenance technique de l'application.

L'application couvre la gestion des clients, la facturation, les réservations d'espaces, les inscriptions aux camps et formations, ainsi que le suivi de la caisse et des dépenses.

2. Prérequis

Installation avec Docker

- Docker Engine 24 ou supérieur
- Docker Compose v2 (intégré à Docker Desktop, ou plugin `docker-compose-plugin` sous Linux)
- 2 Go de mémoire vive disponibles au minimum
- Accès en écriture au dossier du projet

Installation manuelle

- Python 3.12 ou supérieur
- PostgreSQL 14 ou supérieur (ou SQLite pour un essai local)
- pip et venv (fournis avec Python)
- Un serveur Linux, macOS ou Windows avec accès réseau sortant vers PyPI pour l'installation des dépendances

3. Installation avec Docker (recommandée)

Cette méthode démarre l'ensemble des composants (base de données PostgreSQL et application) dans des conteneurs isolés, sans installation de dépendances sur la machine hôte.

Étape 1 — Récupérer le projet

Copier le dossier du projet sur la machine cible, puis s'y placer :

```
cd mbcode
```

Étape 2 — Créer le fichier de configuration

```
cp .env.example .env
```

Ouvrir le fichier `.env` et renseigner au minimum : `DJANGO_SECRET_KEY`, `POSTGRES_PASSWORD`, `ALLOWED_HOSTS`, ainsi que `DJANGO_SUPERUSER_USERNAME` et `DJANGO_SUPERUSER_PASSWORD` pour la création automatique du compte administrateur. Le détail de chaque variable figure à la section 5.

Étape 3 — Construire et démarrer les services

```
docker compose up --build -d
```

Cette commande construit l'image de l'application, démarre la base de données, applique les migrations, collecte les fichiers statiques et crée le compte administrateur si les variables correspondantes sont

renseignées.

Étape 4 — Vérifier le démarrage

```
docker compose logs -f web
```

Le message *« Demarrage de l'application... »* suivi du démarrage de Gunicorn confirme que l'application est opérationnelle.

Étape 5 — Accéder à l'application

Ouvrir un navigateur à l'adresse `http://localhost:8000` (ou l'adresse/le nom de domaine du serveur en production).

Commandes utiles

```
# Arrêter les services (les données sont conservées)
docker compose down

# Arrêter les services et supprimer les données
docker compose down -v

# Exécuter une commande de gestion Django dans le conteneur
docker compose exec web python manage.py <commande>

# Créer un compte administrateur supplémentaire
docker compose exec web python manage.py createsuperuser
```

4. Installation manuelle (sans Docker)

Étape 1 — Créer un environnement virtuel

```
python3 -m venv .venv
source .venv/bin/activate # Sous Windows : .venv\Scripts\activate
```

Étape 2 — Installer les dépendances

```
pip install -r requirements.txt
```

Étape 3 — Configurer l'environnement

```
cp .env.example .env
# Renseigner DATABASE_URL, par exemple :
# DATABASE_URL=postgres://mbcode:motdepasse@localhost:5432/mbcode
# ou laisser vide pour utiliser une base SQLite locale
```

Étape 4 — Appliquer les migrations

```
python manage.py migrate
```

Étape 5 — Créer un compte administrateur

```
python manage.py createsuperuser
```

Étape 6 — Collecter les fichiers statiques

```
python manage.py collectstatic --noinput
```

Étape 7 — Démarrer l'application

```
# Développement
python manage.py runserver

# Production (exemple avec Gunicorn)
gunicorn config.wsgi:application --bind 0.0.0.0:8000 --workers 3
```

En production, l'application doit être placée derrière un serveur web faisant office de reverse proxy (Nginx, par exemple), chargé de la terminaison TLS et du service direct des fichiers volumineux.

5. Configuration des variables d'environnement

Le fichier `.env.example`, fourni à la racine du projet, liste l'ensemble des variables reconnues. Les principales sont résumées ci-dessous.

Variable	Description
`DJANGO_SECRET_KEY`	Clé secrète Django. À générer aléatoirement pour chaque environnement, jamais partagée.
`DEBUG`	True en développement uniquement. Doit être False en production.
`ALLOWED_HOSTS`	Noms d'hôte autorisés à servir l'application, séparés par des virgules.
`CSRF_TRUSTED_ORIGINS`	Origines de confiance pour la protection CSRF (avec le schéma https://).
`DATABASE_URL`	URL de connexion à la base PostgreSQL.
`TIME_ZONE`	Fuseau horaire de l'application (Africa/Douala par défaut).
`POSTGRES_DB` / `USER` / `PASSWORD`	Identifiants utilisés par le conteneur de base de données Docker.
`SECURE_SSL_REDIRECT`	Force la redirection HTTPS lorsque défini à True.
`DJANGO_SUPERUSER_USERNAME`	Identifiant du compte administrateur créé automatiquement au premier démarrage du conteneur.
`DJANGO_SUPERUSER_PASSWORD`	Mot de passe de ce compte. Ne pas laisser la valeur d'exemple en production.
`DJANGO_SUPERUSER_EMAIL`	Adresse e-mail associée à ce compte.

Le fichier `.env` ne doit jamais être partagé ni versionné : il contient des informations sensibles propres à chaque environnement (développement, préproduction, production).

6. Première connexion et création des comptes

Se connecter avec le compte administrateur créé à l'étape précédente sur la page `/connexion/`. Ce compte permet d'accéder à l'ensemble des fonctionnalités, y compris la configuration des espaces, des camps, des formations, des charges fixes et des comptes agents.

Les comptes des agents de vente se créent depuis l'interface d'administration Django, accessible à l'adresse `/admin/`. Un agent peut enregistrer des ventes, des dépenses et des mouvements de caisse, mais ne peut ni modifier ni supprimer un enregistrement déjà créé, ni créer d'espace, de camp ou de formation.

7. Jeu de données de démonstration

Pour découvrir rapidement l'application avec des données représentatives (formateurs, espaces, clients, camp de démonstration, charges fixes), exécuter la commande suivante :

```
# Avec Docker
docker compose exec web python manage.py seed_demo

# Installation manuelle
python manage.py seed_demo
```

Cette commande peut être exécutée plusieurs fois sans créer de doublons.

8. Tâche planifiée : génération des charges fixes

Les charges fixes (loyer, abonnement Internet, etc.) génèrent automatiquement leur dépense mensuelle, mais cette génération doit être déclenchée par une tâche planifiée exécutée une fois par jour :

```
python manage.py generer_charges_fixes
```

Exemple de tâche cron sous Linux (exécution quotidienne à 6h00) :

```
0 6 * * * cd /chemin/vers/mbcode && \
docker compose exec -T web python manage.py generer_charges_fixes
```

Chaque charge fixe ne génère sa dépense qu'une seule fois par mois : la commande peut être exécutée plusieurs fois sans créer de doublon.

9. Sauvegarde et mise à jour

Sauvegarde de la base de données

```
docker compose exec db pg_dump -U mbcode mbcode > sauvegarde_$(date +%F).sql
```

Restauration

```
cat sauvegarde.sql | docker compose exec -T db psql -U mbcode mbcode
```

Mise à jour de l'application

```
docker compose down
# Remplacer les fichiers du projet par la nouvelle version
docker compose up --build -d
```

Les migrations de base de données sont appliquées automatiquement au démarrage du conteneur. Il est recommandé d'effectuer une sauvegarde avant toute mise à jour.

10. Dépannage

L'application ne démarre pas	Consulter les journaux avec `docker compose logs web`. Vérifier que `DATABASE_URL` pointe vers un service accessible.
Erreur « ALLOWED_HOSTS »	Ajouter le nom de domaine ou l'adresse IP utilisée dans la variable `ALLOWED_HOSTS` du fichier `.env`, puis redémarrer.
Page « CSRF verification failed »	Renseigner `CSRF_TRUSTED_ORIGINS` avec l'URL complète (avec https://) utilisée pour accéder à l'application.
Fichiers statiques manquants (page sans mise en forme)	Exécuter `python manage.py collectstatic --noinput` (ou reconstruire le conteneur, cette étape étant automatique avec Docker).

Compte administrateur non créé	Vérifier que `DJANGO_SUPERUSER_USERNAME` et `DJANGO_SUPERUSER_PASSWORD` sont bien renseignés dans `.env`, ou créer le compte manuellement avec `createsuperuser`.

11. Support

Pour toute question relative à l'installation ou au fonctionnement de l'application, contacter l'équipe MBCODE :

MBCODE
Quartier Makepe, Douala, Cameroun
Téléphone : +237 6 59 17 27 62
Email : infos@mbcode.cm